

UNOESC — Universidade do Oeste de Santa Catarina
Desenvolvimento Mobile

RELATÓRIO TÉCNICO

Conexão & Tradição

Aplicativo mobile Android para conexão entre produtores rurais e a comunidade em torno do carneamento comunitário

Trabalho Atividade Avaliativa 2 — Implementação do Aplicativo

Autor: Fabio Costa

Setembro de 2026

1. Introdução

O Conexão & Tradição é um aplicativo mobile Android, desenvolvido em Kotlin, que conecta produtores rurais do Sul do Brasil (Paraná, Santa Catarina e Rio Grande do Sul) à população em geral em torno da tradição do carneamento comunitário de gado e porco. Antigamente, vizinhos se reuniam para ajudar no abate e dividir a carne entre si; o aplicativo resgata essa prática, permitindo que produtores rurais anunciem eventos de carneamento — com data, local, tipo de animal, cortes disponíveis e preço por kg — e que qualquer pessoa interessada encontre esses eventos na sua região, reserve os cortes desejados e participe da experiência.

O problema que o app resolve é a falta de um canal direto e organizado entre quem produz e quem consome, hoje dependente de indicação boca a boca. Isso beneficia as duas pontas: carne mais barata e de procedência conhecida para quem compra, e melhor preço (sem intermediários) para quem vende, além de manter viva a tradição cultural do carneamento comunitário e promover a socialização entre a comunidade urbana e a rural.

Este relatório documenta a implementação do aplicativo na etapa AA2, que deu continuidade ao projeto e ao protótipo elaborados na AA1, cobrindo arquitetura, tecnologias utilizadas, funcionalidades implementadas, principais desafios técnicos enfrentados durante o desenvolvimento e a forma como foram resolvidos, e os testes realizados.

2. Objetivos

2.1. Objetivo geral

Implementar, de ponta a ponta, o aplicativo Android nativo Conexão & Tradição projetado na AA1, cobrindo a totalidade dos requisitos funcionais e não funcionais definidos, com integração real a um backend em nuvem (Firebase) e testes end-to-end entre múltiplos usuários.

2.2. Objetivos específicos

- Implementar as 11 telas/fluxos do protótipo da AA1 (cadastro/login, listagem de eventos, detalhes do evento, chat, perfil) com dados reais, não simulados.
- Integrar o aplicativo a uma API/backend real para autenticação, persistência de dados e comunicação em tempo real (Firebase Authentication e Cloud Firestore).
- Aplicar uma arquitetura offline-first, adequada ao contexto de conectividade intermitente do público rural (RNF02).
- Aplicar princípios de UX e design consistente com a identidade visual definida na AA1.
- Testar o aplicativo de ponta a ponta em cenários reais, com duas contas distintas interagindo entre si (produtor e comprador).
- Publicar o código-fonte em um repositório Git público (GitHub) e documentar o processo de desenvolvimento.

3. Requisitos do Sistema

Os requisitos abaixo foram definidos na etapa AA1 e serviram de guia para toda a implementação da AA2. Todos os requisitos funcionais foram implementados e testados de ponta a ponta.

3.1. Requisitos Funcionais (RF)

Código	Descrição
RF01	Cadastro e login de usuário (e-mail/senha ou "Continuar com Google").
RF02	Perfil do usuário: nome, foto, editar perfil e histórico de participações com avaliações.
RF03	Listagem dos próximos eventos de carneamento em cards (data, cidade/UF, animal, cortes, produtor, nota).
RF04	Busca e filtro de eventos por cidade e por produto/subproduto.
RF05	Cadastro de evento pelo produtor: data, local, animal, cortes/subprodutos e preço por kg.
RF06	Detalhes do evento com tabela de preços por corte e seleção dos itens de interesse.
RF07	Agendamento de participação ("Agendar Carneação"), reservando os cortes escolhidos.
RF08	Chat direto entre comprador e produtor ("Chamar Produtor").
RF09	Localização exata do evento, liberada após confirmação de presença.
RF10	Avaliação pós-evento: produtor finaliza o evento, avalia cada comprador e vice-versa, com fotos da carneação.
RF11	Notificações de confirmação, lembretes e novas mensagens no chat.

3.2. Requisitos Não Funcionais (RNF)

Código	Descrição
RNF01	Usabilidade acessível a públicos com familiaridade digital muito distinta.
RNF02	Tolerância a conectividade intermitente (offline-first, cache local com Room).
RNF03	Baixo consumo de dados (imagens comprimidas, requisições otimizadas).
RNF04	Compatibilidade a partir do Android 8.0 (API 26).
RNF05	Segurança e privacidade: localização exata só após confirmação de presença; LGPD.
RNF06	Desempenho: listagem de eventos carregando em até 3 segundos em rede 3G/4G.
RNF07	Acessibilidade: fontes ajustáveis e alto contraste.
RNF08	Testabilidade quanto a usabilidade, funcionalidade, rede instável, compatibilidade e sincronismo.

4. Justificativa da Plataforma Escolhida

A escolha foi por desenvolvimento nativo para Android, utilizando Kotlin e o ecossistema Android Studio (Google Maps SDK/geolocalização, Firebase e Room para persistência local). Essa decisão se apoia em três fatores centrais:

- Alcance do público-alvo: o Android detém cerca de 85% do mercado de smartphones no Brasil, presente em aparelhos básicos, intermediários e de ponta — essencial para um app que depende da adesão de produtores rurais com celulares mais simples, diferente de um público de nicho de maior poder aquisitivo.
- Conectividade intermitente: boa parte dos eventos de carneamento ocorre em propriedades rurais com sinal de internet limitado. O desenvolvimento nativo permite melhor controle sobre cache local (Room/SQLite) e sincronização em segundo plano do que uma solução híbrida.
- Recursos nativos necessários: geolocalização e mapas (RF03, RF09), notificações (RF11), câmera para fotos do evento e armazenamento local eficiente (RNF02) — todos com acesso completo às APIs nativas, com melhor desempenho e menor consumo de bateria/dados.

A principal desvantagem assumida conscientemente é a maior fragmentação de versões e fabricantes do Android, que exige mais esforço de testes de compatibilidade (RNF08) — custo considerado inferior ao ganho de alcance ao público rural, essencial para a proposta de valor do app.

5. Arquitetura e Tecnologias Utilizadas

5.1. Stack tecnológica

- Linguagem: Kotlin.
- IDE/build: Android Studio, Gradle (Kotlin DSL).
- Backend: Firebase — Authentication (e-mail/senha e Google), Cloud Firestore (banco de dados em tempo real) e notificações locais reagindo ao Firestore.
- Persistência local: Room (SQLite), com estratégia offline-first.
- Arquitetura de camadas: MVVM (Model-View-ViewModel) com uma camada de Repository entre a UI e as fontes de dados (Room + Firestore), seguindo os padrões estudados na disciplina.
- Localização: FusedLocationProviderClient (Google Play Services).
- Seleção de imagens: Android Photo Picker (sem exigir permissão de galeria).

5.2. Estratégia offline-first (RNF02)

Todas as telas de listagem (eventos, participações, perfil) leem primeiro do banco local Room, que funciona como fonte de verdade para a interface — o app nunca fica "em branco" por falta de conexão. Operações de escrita (agendar carneação, enviar mensagem, avaliar) gravam simultaneamente no Room e no Firestore, com o envio ao Firestore tratado como best-effort: se a gravação remota falhar por falta de conexão, a operação local não é desfeita, e uma rotina de sincronização (executada a cada abertura do app) busca do Firestore o que ainda não está refletido localmente assim que a conexão volta.

Esse padrão evoluiu ao longo do desenvolvimento: um mesmo tipo de falha (dado gravado localmente, mas nunca buscado de volta do Firestore após a recriação do cache local) foi identificado e corrigido em três contextos distintos — perfil do usuário, participações em eventos e, mais adiante, a leitura

de participações/usuários de terceiros pelo produtor ao finalizar um evento e avaliar compradores. O item 8 detalha o caso mais elaborado dessa família de bugs.

5.3. Decisão de arquitetura: fotos do evento sem Firebase Storage

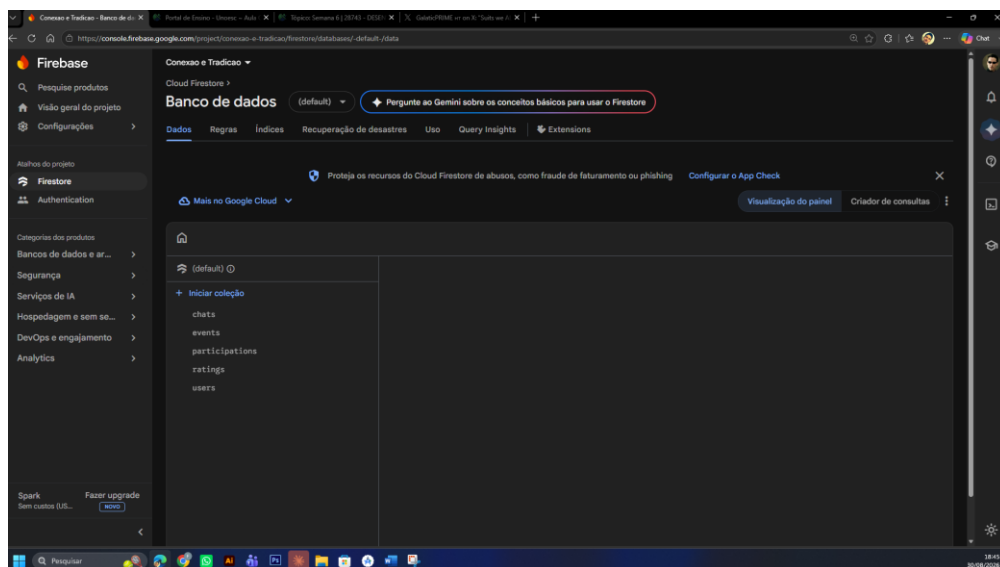
Para a funcionalidade de "Finalizar Evento" com fotos da carneação (RF10), a opção mais natural seria o Firebase Storage. Entretanto, desde outubro de 2024 o Firebase exige upgrade para o plano pago Blaze (com cadastro de cartão de crédito) para habilitar o Storage em projetos novos — mesmo dentro de uma cota gratuita generosa, uma barreira indesejada para um trabalho acadêmico. A alternativa adotada foi salvar as fotos comprimidas (redimensionadas para no máximo 800px, JPEG qualidade 60%) e codificadas em Base64 diretamente como campo de texto nos documentos do Firestore, evitando qualquer configuração adicional no Firebase Console.

6. Modelagem de Dados

O modelo de dados é espelhado entre o Firestore (fonte de verdade remota) e o Room (cache local), com as seguintes entidades principais:

- User — id, nome, e-mail, foto, papel (produtor/comprador), média e contagem de avaliações em estrelas.
- Event — id, produtor, data, cidade/UF, tipo de animal, localização (lat/lng), nota média do produtor, estado de finalização e lista de fotos (Base64) da carneação realizada.
- Cut — corte/subproduto disponível em um evento, com nome e preço por kg.
- Participation — vínculo entre um usuário e um evento: cortes selecionados, status (AGENDADO/CONCLUIDO/CANCELADO) e flags de avaliação já realizada em cada sentido.
- Rating — avaliação em estrelas de um usuário sobre outro, associada a um evento.
- ChatMessage — mensagens trocadas entre produtor e comprador em uma conversa por evento, com remetente, texto e horário.

Essa modelagem permitiu reutilizar as mesmas entidades tanto para a listagem de eventos e o histórico de participações (RF02, RF03) quanto para a avaliação pós-evento nos dois sentidos (RF10), sem duplicar estrutura de dados.



7. Funcionalidades Implementadas

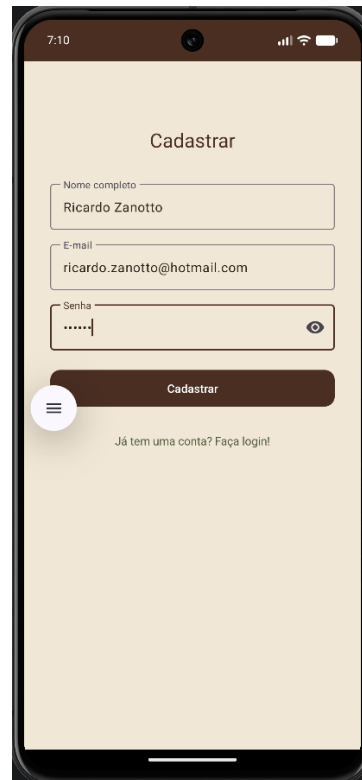
Todas as telas previstas no protótipo da AA1 foram implementadas com dados reais (Firebase), substituindo por completo os mockups estáticos. A seguir, um resumo por tela/fluxo:

7.1. Cadastro e Login (RF01)

Tela de entrada com cadastro/login por e-mail e senha, além de "Continuar com Google" via GoogleSignInClient/Firebase Authentication. O documento de perfil do usuário é criado no Firestore automaticamente no primeiro acesso e sincronizado para o cache local (Room).



Tela Inicial para realização de Login ou cadastro.



Tela para realização de cadastro.

7.2. Listagem de Eventos / Home (RF03, RF04)

Tela inicial com busca por cidade e por produto/subproduto, e uma lista de cards de eventos próximos — data em destaque, cidade/UF, ícone do tipo de animal, cortes disponíveis, produtor e sua nota em estrelas. Eventos já agendados pelo próprio usuário aparecem destacados visualmente (card em verde claro); eventos finalizados pelo produtor não aparecem mais nessa listagem, apenas no histórico correspondente.



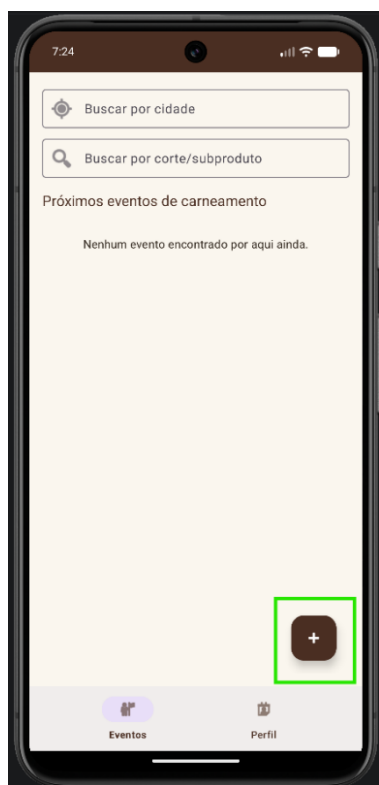
Tela inicial após login, verde são Eventos já inscrito .



Evento terminado que já participou.

7.3. Cadastro de Evento pelo Produtor (RF05) e Detalhes do Evento (RF06, RF07, RF09)

O produtor cadastra um evento informando data, cidade/UF, tipo de animal e a lista de cortes com preço por kg, com opção de capturar a localização exata via GPS do dispositivo. Na tela de detalhes, o comprador vê a tabela de preços por corte, seleciona os itens de interesse e confirma presença pelo botão "Agendar Carneação", que reserva os cortes escolhidos (RF07). A localização exata do evento só é revelada depois da confirmação de presença (RNF05), reaberta em um app de mapas do próprio dispositivo.



Botão na tela inicial para criar Eventos.



Tela para realização de cadastro do evento.

7.4. Chat com o Produtor (RF08) e Notificações (RF11)

Conversa em tempo real entre comprador e produtor, por evento, usando o Firestore como canal de mensagens (leitura em tempo real via listener). Mensagens do produtor são identificadas com o rótulo "Anfitrião" e uma cor de balão diferenciada. Como o plano gratuito do Firebase não permite Cloud Functions com saída de rede — o caminho padrão para notificações push disparadas por servidor —, a solução adotada foi o próprio aplicativo escutar o Firestore em tempo real e disparar a notificação localmente quando chega uma mensagem nova em uma conversa que não está com a tela aberta, com o mesmo resultado prático para o usuário, sem custo de infraestrutura adicional.



Chat entre produtora e comprador.



Notificação.

7.5. Perfil e Avaliação Pós-Evento (RF02, RF10)

A tela de Perfil reúne dados do usuário, edição de nome, a lista de próximos eventos agendados e o histórico de participações concluídas. O produtor conta ainda com a seção "Meus Eventos Finalizados", usada para retornar a um evento já realizado e continuar avaliando compradores pendentes ou revisar as fotos da carneação.

A funcionalidade de avaliação pós-evento (RF10) foi a mais elaborada da AA2: o produtor pode "Finalizar Evento" (com opção de anexar até três fotos da carneação), o que conclui a participação de todos os compradores confirmados e libera a avaliação em estrelas nos dois sentidos — produtor avalia cada comprador, e cada comprador avalia o produtor. A nota é agregada de forma incremental na média exibida tanto no perfil de cada usuário quanto em todos os eventos futuros daquele produtor. As capturas de tela

abaixo mostram o resultado final, testado com duas contas reais (produtor "Neo F2" e comprador "Fabiano Tiger"), confirmando a nota persistindo corretamente nos dois perfis após uma avaliação recíproca:



Comprador ("Fabiano Tiger"): histórico concluído e nota ★ 3,7 (3 avaliações).



Produtor ("Neo F2"): "Meus Eventos Finalizados" e nota ★ 5,0 (2 avaliações).

8. Principais Desafios Técnicos e Soluções

Esta seção documenta os desafios técnicos mais relevantes enfrentados durante a implementação, priorizando os que exigiram investigação mais aprofundada — considerados de maior valor para o relatório por evidenciarem o processo real de depuração de um sistema distribuído (app + backend em nuvem), e não apenas o resultado final.

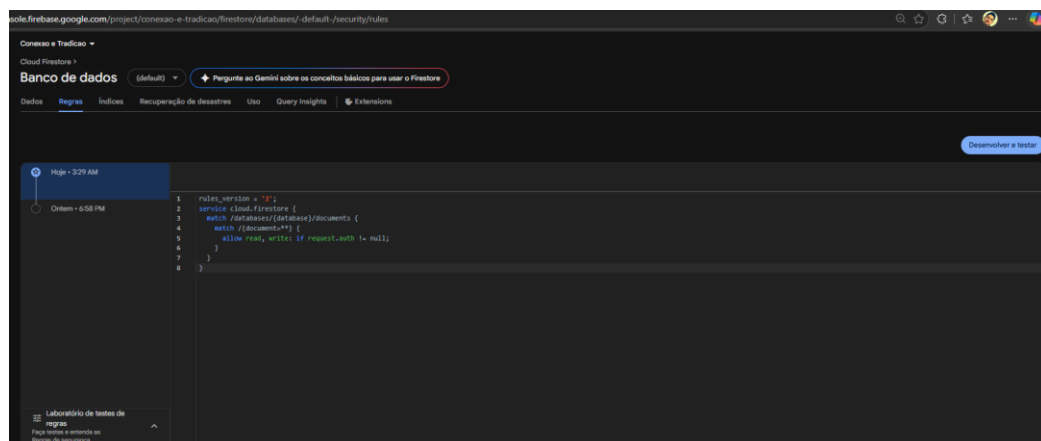
8.1. Sincronização local-remoto em um app offline-first

Um mesmo padrão de falha se repetiu em três momentos distintos do desenvolvimento: um dado era gravado com sucesso no Firestore (nuvem), mas o cache local (Room) do dispositivo que precisava exibi-lo nunca era atualizado de volta — porque a única rotina de sincronização existente buscava apenas os dados do próprio usuário logado, não de terceiros. O caso mais elaborado apareceu na funcionalidade de avaliação (RF10): no dispositivo do produtor, o Room nunca continha as participações nem os perfis dos compradores (apenas os do próprio produtor), então operações como "concluir a participação de um comprador" ou "calcular a nova média de estrelas dele" liam um cache sempre vazio e simplesmente não faziam nada — sem gerar nenhum erro visível, o que tornou o diagnóstico particularmente difícil.

A causa raiz foi confirmada de forma inequívoca instrumentando o código com logs de depuração (tag dedicada, lida via Logcat do Android Studio durante um teste real), que revelaram que a gravação no Firestore sempre funcionava — o problema estava inteiramente do lado da leitura local. A solução aplicada de forma sistemática foi substituir leituras "local-only" por buscas diretas ao Firestore sempre que o dado não fosse encontrado no cache local, com o cache sendo então preenchido a partir dessa busca (evitando repetir a mesma busca remota depois).

8.2. Regras de segurança do Firestore em operações entre usuários diferentes

A funcionalidade de avaliação pós-evento exige, por natureza, que uma conta escreva em documentos pertencentes a outra (o produtor grava a nota do comprador, e vice-versa). As regras de segurança padrão do Firestore, mais restritivas, bloqueiam esse tipo de escrita cruzada. A solução adotada — adequada ao escopo de um trabalho acadêmico, sem necessidade de Cloud Functions — foi ajustar as regras do Firestore para liberar leitura e escrita a qualquer usuário autenticado (não anônimo), preservando a exigência de login, mas sem a granularidade de "cada um só escreve no próprio documento" que um app em produção exigiria.



Regra de segurança publicada no Firebase Console (Firestore Database → Regras).

8.3. Bugs de interface e consistência visual

- Overlays de constraint em XML (ausência de namespace app:/tools: em três layouts) identificados via erro real de build.
- Card de evento sobrepondo elementos e Barrier mal referenciado, corrigidos com âncoras (Barrier) explícitas.
- Contraste insuficiente entre a arte da logo e o fundo padrão do app, corrigido trocando a cor de fundo do ícone do launcher e do card de login.
- RecyclerView "esquecendo" visualmente itens marcados ao reciclar views durante a rolagem — corrigido guardando o estado de seleção fora da view reciclável.

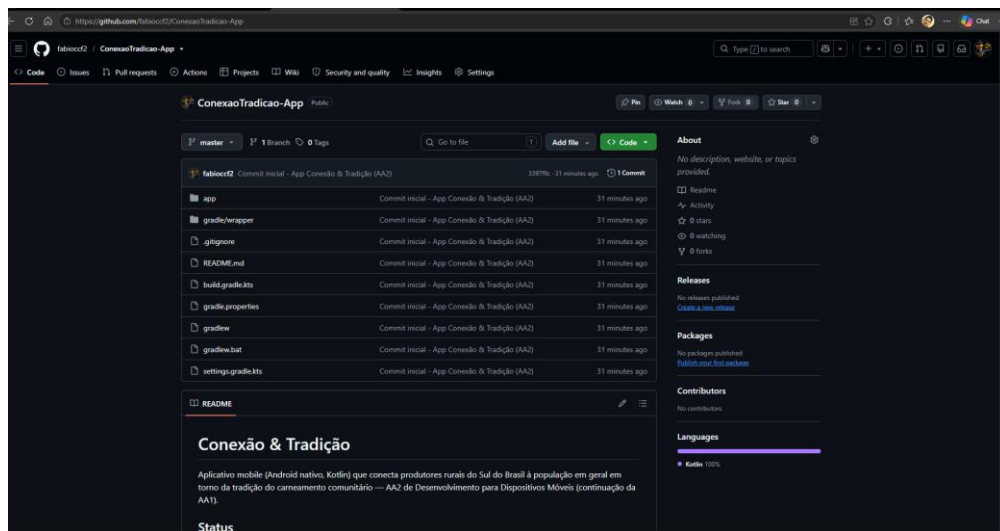
9. Testes Realizados

Além dos testes unitários de tela isolada durante o desenvolvimento, os fluxos críticos foram validados em cenários realistas de uso, com dois emuladores Android rodando simultaneamente (Pixel 7 e Pixel 8), cada um autenticado com uma conta Firebase distinta representando um produtor e um comprador:

- Chat em tempo real (RF08): mensagem enviada de um emulador aparece quase instantaneamente no outro, com a resposta também em tempo real.
- Notificações (RF11): mensagem nova dispara notificação do sistema quando a conversa está fechada, e não dispara quando já está aberta na tela (comportamento intencional).
- Localização (RF09): produtor captura localização via GPS simulado do emulador; comprador recebe o bloqueio antes de agendar e o mapa exato depois de confirmar presença.
- Avaliação pós-evento (RF10): fluxo completo agendar → finalizar evento com fotos → avaliar nos dois sentidos → nota refletida nos dois perfis, validado de ponta a ponta após a correção descrita no item 8.1.
- Login com Google e login por e-mail/senha, incluindo o caso de reabertura do app com sessão já autenticada e banco local recriado (mudança de versão do schema do Room).

10. Versionamento e Repositório

O código-fonte foi publicado em um repositório Git público no GitHub, criado e enviado diretamente pelo recurso nativo do Android Studio (VCS → Share Project on GitHub), incluindo um .gitignore gerado automaticamente para excluir artefatos de build do controle de versão.



Repositório publicado: github.com/fabioccf2/ConexaoTradicao-App

Repositório: <https://github.com/fabioccf2/ConexaoTradicao-App>

11. Conclusão

A implementação da AA2 cobriu a totalidade dos requisitos funcionais definidos na AA1 — desde o cadastro/login até a avaliação pós-evento em estrelas nos dois sentidos —, com integração real a um backend em nuvem (Firebase) e arquitetura offline-first adequada ao contexto de conectividade intermitente do público rural. O desenvolvimento envolveu não apenas a implementação das telas previstas no protótipo, mas também a investigação e correção de falhas de sincronização típicas de sistemas distribuídos, evidenciando na prática conceitos de arquitetura cliente-servidor, cache local e consistência eventual estudados na disciplina.

O código-fonte está publicado e versionado no GitHub, restando como próximas etapas a preparação da apresentação final do trabalho e refinamentos opcionais de menor prioridade (substituição de ícones utilitários genéricos por versões customizadas e remoção de arquivos mortos do projeto).

Referências

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

FIREBASE. Documentação oficial do Firebase (Authentication, Cloud Firestore). Google, 2026. Disponível em: <https://firebase.google.com/docs>.

ANDROID DEVELOPERS. Guia de arquitetura de apps Android (MVVM, Room, offline-first). Google, 2026. Disponível em: <https://developer.android.com/topic/architecture>.